Promptista projektiksi – toteutussuunnitelma

> Tämä tiedosto on projektin "totuuden lähde". Claude Code lukee sen ennen toteutusta, ja muutokset suunnitelmaan tehdään tänne.

1. Tavoite

Pelillistetty verkkosivusto, jolla täysi aloittelija – joku, joka ei tiedä mikä komentorivi on – saa AI-vibekoodausympäristön (Claude Code) pystyyn ja tekee ensimmäisen oman projektinsa.

Periaatteet:

- **Lyhyesti ja selkeästi.** Jokainen tietopaketti selittää vain sen, mitä tarvitaan eteenpäin pääsemiseen. Syventävä tieto on linkkeinä ja avattavina "Lue lisää" -osioina.
- **Tekemällä oppiminen.** Jokaisessa aihealueessa on jotain tehtävää: komento, simulaattoriharjoitus tai visa.
- **Ei kirjautumista.** Edistyminen tallennetaan selaimeen (localStorage).
- **Laajennettava.** Uusi oppimispolku (esim. Lovable) lisätään sisältötiedostoina ilman muutoksia sovelluslogiikkaan.

2. Kohdeyleisö ja käyttötilanne

Asia	Päätös
Osaamistaso	Ei aiempaa kokemusta komentorivistä tai ohjelmoinnista
Käyttöjärjestelmät	Windows ja Mac, erilliset ohjeet. Valinta tehdään aloitusnäytössä ja sitä voi vaihtaa myöhemmin.
Laite	Useimmiten tietokone (komennot kopioidaan sivulta), mutta sivun on toimittava myös puhelimella
Kieli	Suomi. Tekstit sisältötiedostoissa, jotta käännös on myöhemmin mahdollinen.
Sävy	Rento sinuttelu, leikkisä mutta ei lapsellinen. Ei ammattijargonia ilman selitystä.

3. Tekniset päätökset

Osa-alue	Valinta	Perustelu
Kehys	Vite + React + TypeScript	Käyttäjän valinta. TypeScript validoi sisällön rakenteen.
Sisältö	MDX-tiedostot + JSON (visat, sanasto, komennot, merkit)	MDX:n sisään voi upottaa komponentteja (`<Command>` , `<Os>` , `<Quiz>` , `<Term>`).
Sisällön validointi	zod-skeemat + build-vaiheen tarkistus	Rikkinäinen viittaus (esim. puuttuva visa) kaataa buildin eikä päädy tuotantoon.
Reititys	HashRouter (`/#/polku/moduuli`)	GitHub Pages ei tue SPA-reitityksen fallbackia.
Tyylit	CSS custom properties yhdessä teematiedostossa (`src/styles/tokens.css`)	Design voidaan vaihtaa koskematta komponentteihin.
Tallennus	localStorage, versioitu skeema	Ei palvelinta, ei kirjautumista
Testit	Vitest	Tallennus, sisällön validointi, simulaattorin komennot
Julkaisu	GitHub Pages GitHub Actionsilla	Ilmainen. Vite `base` asetetaan repon nimen mukaan.
Analytiikka	Ei mitään	Ei evästeitä, ei evästebanneria
Fontit	Itse hostattuina (ei Google Fonts -CDN:ää)	Tietosuoja: ei pyyntöjä kolmansille osapuolille
Lisenssi	MIT	Julkinen repo, muut voivat ehdottaa korjauksia

Reitityksen vaihtoehdot (päätetty HashRouter):

Vaihtoehto	Plussat	Miinukset
HashRouter	Toimii GitHub Pagesissa ilman kikkailua	URL:issa `#` -merkki
BrowserRouter + `404.html` -kikka	Siistit URL:t	Lisää monimutkaisuutta, ensilataus kiertää 404-sivun kautta

4. Kansiorakenne

```
promptista-projektiksi/
├─ SUUNNITELMA.md            # tämä tiedosto
├─ CLAUDE.md                 # ohjeet Claude Codelle tätä repoa varten
├─ CONTRIBUTING.md           # miten uusi polku tai moduuli lisätään
├─ README.md
├─ public/
├─ src/
│  ├─ content/
│  │  ├─ commands.json       # KAIKKI asennuskomennot yhdessä paikassa
│  │  ├─ glossary.json       # sanasto
│  │  ├─ badges.json         # merkit ja niiden ehdot
│  │  └─ paths/
│  │     └─ claude-code/     # yksi oppimispolku
│  │        ├─ path.json     # polun nimi, kuvaus, aihealueiden järjestys
│  │        └─ <aihealue>/<moduuli>/
│  │           ├─ index.mdx  # tietopaketti
│  │           └─ quiz.json  # monivalintakysymykset (valinnainen)
│  ├─ components/            # Command, Os, Quiz, Term, Terminal, ProgressBar, Badge
…
│  ├─ features/
│  │  ├─ progress/           # localStorage, vienti/tuonti
│  │  └─ terminal-sim/       # terminaalisimulaattori
│  ├─ schemas/               # zod-skeemat sisällölle
│  ├─ styles/tokens.css      # design tokenit
│  └─ pages/
└─ .github/workflows/deploy.yml
```

Uuden polun lisääminen (esim. Lovable): luo kansio `src/content/paths/lovable/` + `path.json` + moduulit. Sovellus löytää polun automaattisesti (Viten `import.meta.glob`). Ohje kirjoitetaan `CONTRIBUTING.md`-tiedostoon.

5. Sisältö: polku "Claude Code alusta alkaen"

Suositeltu järjestys näytetään, mutta mihin tahansa aihealueeseen pääsee suoraan. Mitään ei lukita.

0. Perusteet

Moduuli	Sisältö
0.1 Mitä vibekoodaus on?	AI kirjoittaa koodin, sinä ohjaat. Mitä sillä voi tehdä ja missä se ei (vielä) ole hyvä.
0.2 Terminaali ja komennot	Mikä terminaali on. Analogia: tekstiviestikeskustelu tietokoneen kanssa. **Simulaattoriharjoitus.**

Moduuli	Sisältö
0.3 Kansiot ja polut	Mikä on kansio, polku ja "missä kansiossa olen". **Simulaattoriharjoitus.**
0.4 Git ja GitHub	Git = pelin tallennuspisteet, GitHub = pilvi, jossa tallennukset säilyvät.
0.5 Chat vs. AI-agentti	Agentti tekee asioita koneellasi (luo tiedostoja, ajaa komentoja), chat vain vastaa.

1. Asennukset

Moduuli	Windows	Mac
1.1 Terminaali	PowerShell 7:n asennus ja avaaminen	Terminal-sovelluksen avaaminen (ei asennusta)
1.2 Git	Asennus	Asennus (virallisen ohjeen mukainen tapa)
1.3 VS Code	Asennus, miten kansio avataan	Sama
1.4 GitHub-tili	Tilin luonti + kaksivaiheinen tunnistautuminen	Sama
1.5 GitHub CLI	Asennus + `gh auth login`	Sama
1.6 Claude-tilaus	Mitä tilausta tarvitaan. **Ei hintoja sivulle**, vaan linkki viralliseen hinnastoon.	Sama
1.7 Claude Code	Asennus + kirjautuminen	Sama
1.8 Tarkistuslista	Kaikkien työkalujen versiotarkistukset yhdellä sivulla	Sama

Jokaisessa asennusvaiheessa:

1. Lyhyt selitys: mikä tämä on ja miksi sitä tarvitaan (1–3 virkettä).
2. Komento kopioitavana `<Command>`-komponentilla + linkki viralliseen lähteeseen.
3. **Varmistus:** "Aja `<työkalu> --version`. Miltä tulos näytti?" Monivalinnassa on oikea vaihtoehto, tyypillinen virheilmoitus ja "jotain muuta". Väärä tai virhevalinta ohjaa vianmääritykseen.

2. Claude Code -periaatteet

Moduuli	Sisältö
2.1 Ensimmäinen keskustelu	Miten Claude Code käynnistetään kansiossa, miten sen kanssa puhutaan
2.2 Hyvä prompti	Kerro mitä, miksi ja millä rajoilla. Huono vs. hyvä -esimerkkipareja.
2.3 Plan mode	Suunnittele ensin, rakenna sitten
2.4 CLAUDE.md	Projektin "muistilappu" Claudelle
2.5 Luvat ja turvallisuus	Lue mitä hyväksyt. Älä anna salasanoja. API-avaimet `.env` -tiedostoon, joka ei mene GitHubiin.
2.6 Git tallennuspisteinä	Commit ennen isoa muutosta, miten palataan taaksepäin
2.7 Konteksti	Mikä on konteksti, milloin aloitetaan puhdas keskustelu

3. Ensimmäinen projekti

Käyttäjä tekee **yksinkertaisen staattisen verkkosivun (HTML/CSS/JS, ei build-vaihetta, ei Node.js:ää)** Claude Coden avulla.

Moduuli	Sisältö
3.1 Projektikansio	Kansion luonti, `git init`
3.2 Suunnittele	Plan modessa: mitä sivulle tulee
3.3 Rakenna	Claude rakentaa, käyttäjä avaa sivun selaimessa
3.4 GitHubiin	Commit + uusi repo + push (`gh` -työkalulla)
★ 3.5 Bonus: julkaise	Valinnainen lisätehtävä: GitHub Pages, oma osoite nettiin

Aina saatavilla (ei osa etenemistä)

- **Sanasto:** kaikki termit lyhyesti (1–2 virkettä) + "Lue lisää" -linkki.
- **Vianmääritys:** yleisimmät ongelmat käyttöjärjestelmittäin (esim. "komentoa ei tunnisteta", PowerShellin suoritusoikeudet, terminaali pitää käynnistää uudelleen asennuksen jälkeen).

6. Komponentit

Komponentti	Toiminta
`<Os only="windows|mac">`	Näyttää sisällön vain valitulle käyttöjärjestelmälle
`<Command id="…" />`	Hakee komennon `commands.json`:sta valitun käyttöjärjestelmän mukaan. Kopioi-painike, "Kopioitu!"-palaute, lähdelinkki.
`<Term id="…">teksti</Term>`	Termi alleviivattuna. Hover (kone) / napautus (mobiili) näyttää lyhyen selityksen, linkki sanastoon.
`<Quiz />`	Lataa moduulin `quiz.json`:n. Välitön palaute **jokaiselle vaihtoehdolle** (miksi oikein/väärin). Uudelleenyritys sallittu.
`<Terminal mission="…" />`	Terminaalisimulaattori (ks. luku 7)
`<ReadMore>`	Avattava syventävä osio
`<VerifyStep />`	Asennuksen varmistus monivalintana, virhevalinta linkittää vianmääritykseen

`commands.json`-rakenne (esimerkki):

```
{
  "install-git": {
    "description": "Asentaa Gitin",
    "windows": "…",
    "mac": "…",
    "source": "https://…",
    "verifiedAt": "2026-10-xx"
  }
}
```

Kaikki komennot ovat tässä yhdessä tiedostossa, joten päivitys on yksi muutos yhteen paikkaan. `verifiedAt` kertoo, milloin komento on viimeksi tarkistettu.

7. Terminaalisimulaattori

- Turvallinen harjoitusympäristö selaimessa, ei oikeaa tiedostojärjestelmää.
- Kehote valitun käyttöjärjestelmän mukaan (Windows: PowerShell-tyylinen, Mac: zsh-tyylinen).
- Muistissa toimiva virtuaalinen kansiorakenne.
- Tuetut komennot: `help`, `pwd`, `ls` (+ `dir` Windowsilla), `cd`, `cd ..`, `mkdir`, `clear`, sekä "feikki" versiotarkistukset (`git --version`, `claude --version`).

- **Missiot:** esim. "Luo kansio nimeltä `projekti` ja siirry sinne." Onnistuminen tunnistetaan virtuaalisen tilan perusteella, ja siitä tulee merkki.
- Tuntematon komento → ystävällinen virheviesti + vihje.
- Komentohistoria nuolinäppäimillä. Mobiilissa toimiva syöttökenttä.

8. Pelillistäminen

Mekaniikka	Toteutus
Edistymispalkki	Per aihealue ja koko polku
Merkit	Määritelty `badges.json` :ssa ehtoineen. Esimerkkejä: *Ensimmäinen komento*, *Kansiomestari*, *Asennukset kunnossa*, *Promptiseppä*, *Ensimmäinen commit*, *Julkaistu!* (bonus)
Merkin saaminen	Pieni animaatio + ilmoitus. Kunnioittaa `prefers-reduced-motion` -asetusta.
Moduulin tila	Ei aloitettu / kesken / valmis. Valmis = tietopaketti luettu ja mahdollinen visa läpäisty.
Jatka siitä mihin jäit	Etusivun pääpainike vie seuraavaan keskeneräiseen moduuliin

9. Tallennus

- localStorage-avain `pp-progress-v1` . Skeema versioitu, ja tulevia versioita varten on migraatiofunktio.
- Tallennetaan: valittu käyttöjärjestelmä, moduulien tila, visavastaukset, merkit, simulaattorin missiot.
- Kaikki luku- ja kirjoitusoperaatiot `try/catch` -lohkoissa. Sivu toimii myös, jos localStorage ei ole käytettävissä (esim. yksityinen selausikkuna); silloin käyttäjälle näytetään huomautus.
- **Vienti/tuonti:** "Siirrä edistyminen toiselle laitteelle" → kopioitava koodi (base64-JSON) tai ladattava tiedosto. Tuonnissa koodi validoidaan zodilla.
- **Nollaus:** "Aloita alusta" -painike vahvistuskysymyksellä.

10. Saavutettavuus ja laatu

- WCAG 2.1 AA: kontrastit, näppäimistökäyttö, näkyvä fokus, `aria-live` visapalautteelle ja kopioi-ilmoitukselle.
- Mobiili: toimii 360 px leveydestä alkaen, ei vaakavieritystä (paitsi koodilohkoissa).
- Vaalea ja tumma teema (`prefers-color-scheme` + manuaalinen vaihto).
- Lighthouse: tavoite 90+ kaikissa kategorioissa.

11. Design

Ulkoasu suunnitellaan erikseen. Toteutus tehdään aluksi **neutraalilla, siistillä placeholder-teemalla**, ja kaikki värit, fontit, välistykset ja pyöristykset tulevat `tokens.css`-tiedostosta.

Design-brief (myöhempää design-promptia varten):

- Kohdeyleisö on tavallinen ihminen, ei ohjelmoija. **Ei terminaalihenkistä tai "hakkerimaista" ulkoasua.**
- Tunnelma: ystävällinen, rohkaiseva, selkeä – kuin hyvä verkkokurssi tai kielenoppimissovellus.
- Komentolohkot saavat näyttää teknisiltä, mutta niiden ympäristö ei.
- Designin on tuotettava ainakin: väripaletti (vaalea + tumma), typografia, välistysasteikko, pyöristykset, varjot sekä komponenttien ulkoasu (painike, kortti, komentolohko, visa, merkki, edistymispalkki, terminaalisimulaattori).

12. Sisällön kirjoittaminen ja tarkistus

- **Asennuskomennot ja -ohjeet tarkistetaan virallisista lähteistä toteutushetkellä** (Anthropicin Claude Code -dokumentaatio, git-scm.com, cli.github.com, Microsoftin PowerShell-dokumentaatio, code.visualstudio.com). Komentoja ei kirjoiteta muistista.
- Hintoja tai tilausten yksityiskohtia ei kirjoiteta sivulle, vaan niihin linkitetään.
- Tietopaketti: enintään noin 150–250 sanaa + komennot/harjoitus. Pidempi sisältö `<ReadMore>`-osioon.
- Visat: 2–4 kysymystä per moduuli (ei jokaisessa moduulissa), 3–4 vaihtoehtoa, jokaiselle vaihtoehdolle selitys.
- Kieli: hyvä yleiskieli, rento sinuttelu, ei anglismeja. Termit esitellään ensimmäisellä kerralla `<Term>`-komponentilla.

13. Toteutusvaiheet

Jokaisen vaiheen lopuksi: testit ja build läpi → commit.

Vaihe	Sisältö
1	Projektin pohja: Vite + React + TS, ESLint, Vitest, kansiorakenne, `CLAUDE.md`, `tokens.css`
2	Sisältöskeemat (zod), sisällön lataus `import.meta.glob`illa, build-vaiheen validointi
3	Tallennus (`progress`-feature) + testit, vienti/tuonti/nollaus
4	Peruskomponentit: `Os`, `Command`, `Term`, `ReadMore`, `Quiz`, `VerifyStep`
5	Sivut: etusivu + käyttöjärjestelmän valinta, polku, moduuli, sanasto, vianmääritys, asetukset

Vaihe	Sisältö
6	Terminaalisimulaattori + missiot + testit
7	Merkit ja edistymispalkit
8	Sisältö: aihealue 0 → 3, komennot tarkistettuina virallisista lähteistä
9	Saavutettavuus- ja mobiilitarkistus, Lighthouse
10	GitHub Actions -julkaisu GitHub Pagesiin, `README.md`, `CONTRIBUTING.md`, `LICENSE`

14. Rajattu pois (toistaiseksi)

- Kirjautuminen ja palvelinpuoli
- Analytiikka
- Muut kielet kuin suomi (valmius on olemassa)
- Linux-ohjeet
- Muut oppimispolut (Lovable ym.) – arkkitehtuuri tukee niitä